

Nanhu OP-TEE 自治 IRQ 处理测试方案

本文档描述下一阶段要实现和验证的目标：

1.

hart 已经在 OP-TEE 安全世界中运行时，发生安全中断，由 OP-TEE 自己 claim、handler、complete。
2.

hart 已经在 OP-TEE 安全世界中运行时，发生非安全中断，由 OP-TEE 自己 claim 后直接丢弃，不返回 REE，不延后给 Linux。

这里说的“安全世界中”不是旧的 42/43、44/45 路径。旧路径都是从 REE/Linux 触发开始，安全中断先进入 OpenSBI，然后 OpenSBI 再切到 OP-TEE。新目标要求中断处理全程在 OP-TEE 内完成，不能依赖 REE/Linux handler，也不能依赖 OpenSBI 的 secure IRQ relay 来完成 claim/complete。

当前已有测试与新目标的边界

旧测试 42/43

资源：

- IRQ42: Non-secure, MMIO 0x30000000
- IRQ43: Secure, MMIO 0x30001000

流程：

```
Linux devmem 触发 IRQ42
Linux IRQ42 handler 内触发 IRQ43
IRQ43 进入 OpenSBI M-context
OpenSBI claim IRQ43
OpenSBI 切入 OP-TEE FIQ 入口
OpenSBI 在 OP-TEE 返回后 complete IRQ43
Linux IRQ42 handler 继续执行
```

验证目标：

```
REE 下安全中断抢占非安全中断
```

旧测试 44/45

资源：

- IRQ44: Non-secure, MMIO 0x30002000
- IRQ45: Secure, MMIO 0x30003000

流程：

```
Linux devmem 触发 IRQ45
IRQ45 进入 OpenSBI M-context
OpenSBI claim IRQ45
OpenSBI 在进入 OP-TEE FIQ 前设置 ws=1
OpenSBI 测试 hook 触发 IRQ44
观察 IRQ44 是否抢占 IRQ45
OpenSBI 在 OP-TEE 返回后 complete IRQ45
```

验证目标：

```
secure IRQ45 处理中发生 NS IRQ44 , NS IRQ44 不能抢占 secure IRQ45
```

注意：这个测试里的 IRQ44 是 OpenSBI hook 触发的，不是 OP-TEE 主动触发的。因此它不能证明“OP-TEE 安全世界内自行处理/丢弃非安全中断”。

新目标的正确架构

新目标需要这条路径：

```
Linux 用户态 CA
-> TEEC_InvokeCommand()
-> Linux OP-TEE driver
-> OpenSBI MPXY 普通 OP-TEE 调用入口
-> OP-TEE PTA 正在运行
-> PTA 写 irqgen MMIO 触发 IRQ44 或 IRQ45
-> PLIC 把外部中断递交给 OP-TEE S-context
-> OP-TEE trap/vector 进入 native external interrupt path
-> OP-TEE PLIC claim
-> OP-TEE 根据 sec_src 判断安全属性
    - secure IRQ : 调用 OP-TEE handler , 然后 complete
    - non-secure IRQ : 打印 drop , 然后 complete , 不调用 Linux , 不保留 pending
```

这条路径里，OpenSBI 只负责普通 world switch/MPXY 进入和退出 OP-TEE，不负责本次 IRQ 的 claim/complete。

关键方案决策

决策 1：安全世界内 IRQ 由 OP-TEE claim/complete

这里选择 OP-TEE claim/complete，而不是继续让 OpenSBI claim/complete。

原因：

- 新目标定义的是“hart 已经在 OP-TEE 安全世界里运行”，因此 IRQ owner 应该是 OP-TEE 的 S-context。
- 如果仍让 OpenSBI claim/complete，本质还是旧 secure IRQ relay，只是触发点换成了 PTA，不能证明 OP-TEE 自治处理。

- OP-TEE 自己 claim 后才能做到“secure IRQ 调 handler，non-secure IRQ 直接 drop”这两个动作都在 OP-TEE 内完成。

为避免 OpenSBI 和 OP-TEE 重复 claim，需要配套修改 PLIC 路由：

```
ws=1 时：
S-context 接收 IRQ44/IRQ45
M-context 不接收 IRQ44/IRQ45
```

这样 IRQ 到来时只有 OP-TEE S-context 能 claim，OpenSBI M-context 不会同时 claim。

决策 2：OP-TEE 内的 non-secure IRQ 不是“不递交”，而是“递交后丢弃”

这点很重要。要验证“安全世界发生非安全中断，直接丢掉”，有两种语义：

```
PLIC 直接不递交给任何 context
OP-TEE 收到、claim、判断 sec_src=0、complete/drop
```

本文档采用第二种。原因是测试需要可观测、可证明：

- 如果 PLIC 直接不递交，日志上只能看到“什么都没发生”，很难区分是正确屏蔽、没触发、没 enable，还是路由错误。
- OP-TEE claim 后打印 drop，可以明确证明“非安全中断确实到达安全世界策略层，并被安全世界丢弃”。
- complete/drop 后 PLIC 状态清干净，不会留下 pending/in_service 污染后续测试。

因此新目标的 non-secure IRQ44 预期不是 Linux 处理，也不是 OpenSBI 处理，而是：

```
QEMU PLIC S-context claim irq=44 sec=0 ws=1
OP-TEE PLIC drop irq=44
QEMU PLIC complete irq=44
```

决策 3：触发入口用 PTA/CA，而不是 OP-TEE shell

OP-TEE 没有交互式 shell，所以“进入安全世界后触发 IRQ”的正确方式是：

```
REE 用户态 CA 发起 TEEC_InvokeCommand
OP-TEE PTA command 正在执行
PTA 内写 irqgen MMIO
IRQ 在 PTA 执行窗口内打进 OP-TEE
```

这和 `xtest` 的模式一致：`xtest` 本身运行在 Linux 用户态，但它通过 TEE client API 让 OP-TEE 执行 TA/PTA 代码。我们新增的 `nanhu_irq_test` CA 也是同样的入口，只是命令更小、更聚焦。

必须修改的模块

1. OpenSBI: 普通 MPXY 进入 OP-TEE 时同步 world_state

当前问题:

`gemu/hw/intc/sifive_plic.c` 的 `plic_world_state()` 读取 CPU 的 `ws_csr`。但是当前 OpenSBI 只在 secure IRQ relay 路径里显式设置 `world_state`:

- `opensbi/lib/utils/irqchip/fdt_irqchip_plic.c:191`: secure IRQ 进入 OP-TEE 前设置 `ws=1`
- `opensbi/lib/utils/irqchip/fdt_irqchip_plic.c:233`: FIQ 返回后设置 `ws=0`

普通 CA/PTA 调用路径在:

- `opensbi/lib/utils/mpxy/fdt_mpxy_opteed.c:204`: `sbi_ecall_tee_domain_enter()`
- `opensbi/lib/utils/mpxy/fdt_mpxy_opteed.c:211`: `sbi_ecall_tee_domain_exit()`

这两个函数当前只做 domain enter/exit, 没有同步 PLIC `world_state`。如果不改, PTA 正在运行时 PLIC 仍可能认为 `ws=0`, 那 secure IRQ 会继续进 OpenSBI, NS IRQ 会继续进 Linux, 不符合新目标。

修改方案:

新增一个 OpenSBI PLIC helper:

- 修改 `opensbi/include/sbi_utils/irqchip/fdt_irqchip_plic.h`
- 修改 `opensbi/lib/utils/irqchip/fdt_irqchip_plic.c`

建议接口:

```
void fdt_plic_set_current_world_state(u32 ws);
```

实现逻辑:

```
void fdt_plic_set_current_world_state(u32 ws)
{
    struct sbi_scratch *scratch = sbi_scratch_thishart_ptr();
    struct plic_data *plic = plic_get_hart_data_ptr(scratch);
    long mctx = plic_get_hart_mcontext(scratch);

    if (!plic || mctx < 0)
        return;

    plic_set_world_state(plic, mctx, ws);
    sbi_printf("plic-sec: mpxy switch ws -> %u mctx=%ld hart=%u\n",
               ws, mctx, current_hartid());
}
```

然后修改:

- `opensbi/lib/utils/mpxy/fdt_mpxy_opteed.c`

在 `sbi_ecall_tee_domain_enter()` 中:

```
fdt_plic_set_current_world_state(1);
sbi_domain_context_set_mepc(tdomain, entry_point);
sbi_domain_context_enter(tdomain);
```

在 `sbi_ecall_tee_domain_exit()` 中:

```
sbi_domain_context_exit();
fdt_plic_set_current_world_state(0);
```

预期效果:

```
Linux CA 调 PTA
OpenSBI MPXY enter 时打印 ws -> 1
PTA 返回后 OpenSBI MPXY exit 打印 ws -> 0
```

判定标准:

- PTA 运行期间 QEMU PLIC claim 日志必须显示 `ws=1`。
- 如果 PTA 触发 IRQ45 仍看到 `plic-sec: enter tee fiq irq=45`, 说明普通调用期间 `ws` 没有正确保持为 1。

2. QEMU PLIC: secure world 下外部 IRQ 递交给 OP-TEE S-context

当前文件:

- `qemu/hw/intc/sifive_plic.c`

当前关键逻辑:

- `qemu/hw/intc/sifive_plic.c:104: plic_irq_allowed()`
- `qemu/hw/intc/sifive_plic.c:111: S-context` 当前只允许 `sec == ws`
- `qemu/hw/intc/sifive_plic.c:115: M-context` 当前按 `sec != ws` 路由
- `qemu/hw/intc/sifive_plic.c:502: sifive_plic_irq_request()` 设置 pending

为了让“OP-TEE 自己 drop 非安全中断”成立, secure world 下的 NS IRQ 不能被 PLIC 直接挡在 OP-TEE 外面, 也不能交给 M/REE。它必须递交给 OP-TEE S-context, 由 OP-TEE claim 后判断 `sec_src=0` 并 complete/drop。

修改后的路由策略:

```
ws=0, non-secure IRQ -> S-context, Linux 处理
ws=0, secure IRQ    -> M-context, OpenSBI relay 到 OP-TEE
ws=1, secure IRQ    -> S-context, OP-TEE handler 处理
ws=1, non-secure IRQ -> S-context, OP-TEE claim 后 drop
```

也就是说，`ws=1` 时，S-context 是 OP-TEE 的安全世界上下文，安全和非安全外部 IRQ 都先递交给 OP-TEE，由 OP-TEE 决定处理或丢弃。

建议修改 `plic_irq_allowed()`:

```
static bool plic_irq_allowed(SiFivePLICState *plic, uint32_t addrid,
                             uint32_t irq)
{
    PLICMode mode = plic->addr_config[addrid].mode;
    uint32_t sec = plic_sec_src(plic, irq);
    uint32_t ws = plic_world_state(plic, addrid);

    if (mode == PLICMode_S) {
        if (ws)
            return true;          /* OP-TEE receives both S and NS, then
decides */
        return !sec;             /* Linux receives only NS */
    }

    if (mode == PLICMode_M) {
        if (ws)
            return false;         /* secure world owns external IRQ policy */
        return sec;              /* REE world secure IRQ still uses OpenSBI
relay */
    }

    return false;
}
```

保留:

- `qemu/hw/intc/sifive_plic.c:135`: 同一 context `in_service` 时不能重复 claim。
- `qemu/hw/intc/sifive_plic.c:236`: claim 后锁存 `irq_track.req_sec`。
- `qemu/hw/intc/sifive_plic.c:357`: complete 校验安全属性。

建议增加调试日志:

在 `sifive_plic_claimed()` 找到候选 IRQ 后，或 claim read 成功后，已有:

```
plic-tee claim: ctx=... mode=S irq=... sec=... ws=...
```

测试时需要看到:

```
plic-tee claim: ... mode=S irq=44 sec=0 ws=1
plic-tee claim: ... mode=S irq=45 sec=1 ws=1
```

不应该看到：

```
plic-tee claim: ... mode=M irq=44 ...
plic-tee claim: ... mode=M irq=45 ... # 在普通 PTA 触发 secure IRQ 时不应出现
```

3. OP-TEE：把 RISC-V 外部中断从 foreign 改成 native

当前文件：

- `config/plat-nanhu/platform_config.h`

当前配置：

```
#define PLAT_THREAD_EXCP_FOREIGN_INTR \
    (CSR_XIE_EIE | CSR_XIE_TIE | CSR_XIE_SIE)
#define PLAT_THREAD_EXCP_NATIVE_INTR    (0)
```

当前问题：

RISC-V 外部中断 `CSR_XIE_EIE` 被归类为 foreign interrupt。OP-TEE trap 入口会在 `thread_rv.S` 中判断：

- `optee_os/core/arch/riscv/kernel/thread_rv.S:112`：如果 XIP 属于 `THREAD_EXCP_FOREIGN_INTR`
- `optee_os/core/arch/riscv/kernel/thread_rv.S:125`：tail call `thread_foreign_interrupt_handler`
- `optee_os/core/arch/riscv/kernel/thread_rv.S:756`：foreign handler 会 suspend thread
- `optee_os/core/arch/riscv/kernel/thread_rv.S:785`：返回 untrusted domain

这正好和新目标相反。

修改方案：

把外部中断 EIE 改为 native：

```
#define PLAT_THREAD_EXCP_FOREIGN_INTR \
    (CSR_XIE_TIE | CSR_XIE_SIE)
#define PLAT_THREAD_EXCP_NATIVE_INTR \
    (CSR_XIE_EIE)
```

这样 OP-TEE 运行期间收到 S external interrupt 时，会走：

```
thread_trap_vect
-> native_interrupt_from_kernel / native_interrupt_from_user
-> thread_native_interrupt_handler()
```

```
-> IRQ_XEXT  
-> interrupt_main_handler()
```

相关代码：

- `optee_os/core/arch/riscv/kernel/thread_rv.S:127`: native interrupt path
- `optee_os/core/arch/riscv/kernel/thread_arch.c:156`:
`thread_native_interrupt_handler()`
- `optee_os/core/arch/riscv/kernel/thread_arch.c:166`: `IRQ_XEXT`
- `optee_os/core/arch/riscv/kernel/thread_arch.c:151`: `thread_irq_handler()`

预期效果：

PTA 运行期间触发 IRQ44/45 时，OP-TEE 不再返回 REE 处理 foreign interrupt，而是进入 OP-TEE native interrupt path。

4. OP-TEE：Nanhu 平台 `interrupt_main_handler` 改为真正 PLIC dispatch

当前文件：

- `config/plat-nanhu/main.c`

当前状态：

`config/plat-nanhu/main.c:30` 的 `interrupt_main_handler()` 只打印：

```
[optee] vector_fiq_entry begin, secure IRQ handled  
[optee] vector_fiq_entry end
```

这只是旧 OpenSBI relay 测试用 marker，不是真正 OP-TEE PLIC dispatch。

修改方案：

把平台中断入口改成调用 `plic_it_handle()`：

```
void interrupt_main_handler(void)  
{  
    if (IS_ENABLED(CFG_RISCV_PLIC))  
        plic_it_handle();  
}
```

如果还想保留旧 42/43、44/45 OpenSBI relay 的 marker，不建议继续放在通用 `interrupt_main_handler()` 里。更合理的做法是：

- 旧 OpenSBI relay 路径靠 OpenSBI 日志和 OP-TEE FIQ entry 日志判断。
- 新 OP-TEE native path 靠 `optee-plic` 和 IRQ handler 日志判断。

否则一个函数同时承担“OpenSBI relay FIQ marker”和“OP-TEE native PLIC dispatch”，很容易让测试语义再次混淆。

5. OP-TEE PLIC driver: claim 后根据 sec_src 分流

当前文件：

- optee_os/core/drivers/plic.c
- optee_os/core/include/drivers/plic.h

当前函数：

- optee_os/core/drivers/plic.c:142: plic_claim_interrupt()
- optee_os/core/drivers/plic.c:149: plic_complete_interrupt()
- optee_os/core/drivers/plic.c:273: plic_it_handle()

当前 plic_it_handle() 行为：

```
id = claim();
if valid:
    interrupt_call_handlers(id);
complete(id);
```

新目标需要：

```
claim 后先读 sec_src
- sec_src=1: 这是安全中断，调用 handler，然后 complete
- sec_src=0: 这是非安全中断，直接 complete/drop，不调用 handler，不返回 REE
```

需要给 OP-TEE PLIC driver 增加 Nanhu 安全扩展寄存器支持。

QEMU PLIC 安全扩展偏移：

- qemu/hw/intc/sifive_plic.c:35: PLIC_SEC_SRC_BASE = 0x10000
- qemu/hw/intc/sifive_plic.c:36: PLIC_WORLD_STATE_BASE = 0x11000
- qemu/hw/intc/sifive_plic.c:37: PLIC_IRQ_TRACK_BASE = 0x12000

OP-TEE 侧建议新增：

```
#define PLIC_SEC_SRC_OFFSET 0x10000

static bool plic_is_secure_source(struct plic_data *pd, uint32_t source)
{
    return io_read32(pd->plic_base + PLIC_SEC_SRC_OFFSET +
        4 * (source / 32)) & BIT(source % 32);
}
```

然后修改 `plic_it_handle()`:

```
void plic_it_handle(void)
{
    struct plic_data *pd = &plic_data;
    uint32_t id = plic_claim_interrupt(pd);
    bool sec = false;

    if (!id || id > pd->max_it) {
        DMSG("ignoring interrupt %" PRIu32, id);
        return;
    }

    sec = plic_is_secure_source(pd, id);
    IMSG("[optee-plic] claim irq=%" PRIu32 " sec=%u", id, sec);

    if (!sec) {
        IMSG("[optee-plic] drop non-secure irq=%" PRIu32, id);
        plic_complete_interrupt(pd, id);
        IMSG("[optee-plic] complete dropped irq=%" PRIu32, id);
        return;
    }

    interrupt_call_handlers(&pd->chip, id);
    plic_complete_interrupt(pd, id);
    IMSG("[optee-plic] complete secure irq=%" PRIu32, id);
}
```

注意点:

- 非安全中断 drop 必须执行 complete, 否则 PLIC `irq_track` 会一直 in-service。
- 非安全中断 drop 不应调用 `interrupt_call_handlers()`, 否则没有 handler 时 OP-TEE 会 mask 这个 IRQ。
- 安全中断如果没有 handler, `interrupt_call_handlers()` 会 mask unhandled interrupt。测试前必须注册 IRQ45 handler。

6. OP-TEE: 注册安全 IRQ45 handler

当前文件建议:

- `config/plat-nanhu/main.c`

需要在 OP-TEE 初始化阶段注册 IRQ45 handler。

建议定义:

```
#define NANHU_SEC_TEST_IRQ 45
```

handler:

```
static enum itr_return nanhu_secure_irq_handler(struct itr_handler *h
__unused)
{
    IMSG("[optee-test] secure irq45 handler begin");
    IMSG("[optee-test] secure irq45 handler end");
    return ITRR_HANDLED;
}
```

注册逻辑可以放在 `boot_primary_init_intc()` 之后，或者新增一个 platform init helper。

可用接口：

- `optee_os/core/include/kernel/interrupt.h:119: ITR_HANDLER`
- `optee_os/core/include/kernel/interrupt.h:375: interrupt_add_handler_with_chip()`
- `optee_os/core/include/kernel/interrupt.h:230: interrupt_enable()`
- `optee_os/core/kernel/interrupt.c:82: interrupt_call_handlers()`

示意：

```
static struct itr_handler nanhu_sec_irq_hdl =
    ITR_HANDLER(NULL, NANHU_SEC_TEST_IRQ, 0, nanhu_secure_irq_handler,
    NULL);

static void nanhu_secure_irq_test_init(void)
{
    struct itr_chip *chip = interrupt_get_main_chip();
    TEE_Result res = TEE_SUCCESS;

    res = interrupt_add_handler_with_chip(chip, &nanhu_sec_irq_hdl);
    if (res)
        panic("failed to register Nanhu secure IRQ handler");

    interrupt_enable(chip, NANHU_SEC_TEST_IRQ);
    IMSG("[optee-test] secure irq45 handler registered");
}
```

调用位置：

```
void boot_primary_init_intc(void)
{
    plic_init(PLIC_BASE);
    nanhu_secure_irq_test_init();
}
```

注意：如果 `interrupt_get_main_chip()` 在 `plic_init()` 后可用，就可以这样写。`plic_init()` 当前在 `optee_os/core/drivers/plic.c:270` 调用 `interrupt_main_init(&plic_data.chip)`，因此之后可以取 main chip。

7. OP-TEE：新增 PTA 作为安全世界内触发入口

目标：

通过 REE Linux 用户态 CA 进入 OP-TEE PTA。进入后，触发动作发生在 OP-TEE 代码中，而不是 OpenSBI hook 或 Linux IRQ handler 中。

新增文件：

- `optee_os/lib/libutee/include/pta_nanhu_irq_test.h`
- `optee_os/core/pta/nanhu_irq_test.c`

修改文件：

- `optee_os/core/pta/sub.mk`
- `config/plat-nanhu/conf.mk`

头文件建议：

```
#ifndef __PTA_NANHU_IRQ_TEST_H
#define __PTA_NANHU_IRQ_TEST_H

#define PTA_NANHU_IRQ_TEST_UUID \
    { 0x6d99f02a, 0x9f6a, 0x4c30, \
      { 0xa8, 0x4b, 0x4e, 0x21, 0x44, 0x9c, 0x10, 0x01 } }

#define PTA_NANHU_IRQ_TEST_CMD_BUSY      0
#define PTA_NANHU_IRQ_TEST_CMD_TRIGGER_NS 1
#define PTA_NANHU_IRQ_TEST_CMD_TRIGGER_SEC 2

#endif
```

PTA 需要映射 irqgen MMIO：

- NS IRQ44 irqgen：0x30002000
- Secure IRQ45 irqgen：0x30003000

建议在平台配置里定义：

- `config/plat-nanhu/platform_config.h`

```
#define NANHU_IRQGEN_NS2_BASE    0x30002000
#define NANHU_IRQGEN_SEC2_BASE  0x30003000
#define NANHU_IRQGEN_REG_SIZE   0x1000
#define NANHU_IRQGEN_TRIGGER    0x0
```

并在 OP-TEE 注册 MMIO:

可放在 `config/plat-nanhu/main.c` 或一个单独 platform driver:

```
register_phys_mem_pgdir(MEM_AREA_IO_SEC, NANHU_IRQGEN_NS2_BASE, 0x2000);
```

PTA 示例逻辑:

```
static TEE_Result trigger_ns(uint32_t types, TEE_Param
params[TEE_NUM_PARAMS])
{
    vaddr_t va = core_mmu_get_va(NANHU_IRQGEN_NS2_BASE, MEM_AREA_IO_SEC,
                                NANHU_IRQGEN_REG_SIZE);

    if (types != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE, TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE, TEE_PARAM_TYPE_NONE))
        return TEE_ERROR_BAD_PARAMETERS;

    IMSG("[optee-test] trigger ns irq44 begin");
    io_write32(1, va + NANHU_IRQGEN_TRIGGER);
    mdelay(100);
    IMSG("[optee-test] trigger ns irq44 end");
    return TEE_SUCCESS;
}
```

secure 版本:

```
static TEE_Result trigger_sec(uint32_t types, TEE_Param
params[TEE_NUM_PARAMS])
{
    vaddr_t va = core_mmu_get_va(NANHU_IRQGEN_SEC2_BASE, MEM_AREA_IO_SEC,
                                NANHU_IRQGEN_REG_SIZE);

    IMSG("[optee-test] trigger secure irq45 begin");
    io_write32(1, va + NANHU_IRQGEN_TRIGGER);
    mdelay(100);
    IMSG("[optee-test] trigger secure irq45 end");
    return TEE_SUCCESS;
}
```

PTA 注册参考:

- `optee_os/core/pta/tests/invoke.c:403: invoke_command()`
- `optee_os/core/pta/tests/invoke.c:442: pseudo_ta_register()`
- `optee_os/lib/libutee/include/pta_invoke_tests.h:9: UUID/command 定义方式`

构建接入:

- `optee_os/core/pta/sub.mk`

```
srcs-$(CFG_NANHU_IRQ_TEST_PTA) += nanhu_irq_test.c
```

- `config/plat-nanhu/conf.mk`

```
$(call force,CFG_NANHU_IRQ_TEST_PTA,y)
```

8. REE Linux：新增小 CA，类似 xtest 触发 PTA

当前 rootfs 里已经有：

- `/usr/bin/xtest`
- `/usr/sbin/tee-supPLICANT`
- `/usr/lib/libteeec.so`

也就是说用户态 CA 路径是可行的。

建议新增源码目录：

- `host/nanhu_irq_test/nanhu_irq_test.c`
- `host/nanhu_irq_test/Makefile`

CA 行为：

```
nanhu_irq_test busy
nanhu_irq_test trigger-ns
nanhu_irq_test trigger-sec
```

核心调用：

```
TEEC_InitializeContext(NULL, &ctx);
TEEC_OpenSession(&ctx, &sess, &uuid, TEEC_LOGIN_PUBLIC, NULL, NULL,
&origin);
TEEC_InvokeCommand(&sess, cmd, &op, &origin);
TEEC_CloseSession(&sess);
TEEC_FinalizeContext(&ctx);
```

Linux driver 侧调用链：

- `linux/drivers/tee/tee_core.c:830: TEE_IOC_OPEN_SESSION`
- `linux/drivers/tee/tee_core.c:832: TEE_IOC_INVOKE`
- `linux/drivers/tee/tee_core.c:543: tee_ioctl_invoke()`
- `linux/drivers/tee/optee/call.c:362: optee_open_session()`

- `linux/drivers/tee/optee/call.c:512: optee_invoke_func()`
- `linux/drivers/tee/optee/smc_abi.c:1446: RISC-V 下通过 MPXY 发给 OpenSBI/OP-TEE`

CA 编译进 rootfs 的方式有两个：

1. 临时手动交叉编译，然后复制进 `rootfs`，重新 `make rootfs-pack`。
2. 给顶层 Makefile 加 `host/nanhu_irq_test` 编译和安装规则，长期推荐这种。

测试用例

Test 0: 确认 xtest/TEE 基础环境

进入 QEMU Linux：

```
ls -l /dev/tee*
which tee-suppllicant
which xtest
xtest --help
```

预期：

```
/dev/tee0 存在
/usr/sbin/tee-suppllicant 存在
/usr/bin/xtest 存在
```

如果基础 TEE 调用不通，先不要测 IRQ。

Test 1: 普通 OP-TEE PTA 调用期间 world_state 保持为 1

执行：

```
nanhu_irq_test busy
```

预期 OpenSBI 日志：

```
plic-sec: mpxy switch ws -> 1 ...
[optee-test] busy begin
[optee-test] busy end
plic-sec: mpxy switch ws -> 0 ...
```

判定：

- PASS: PTA 运行期间 `ws=1`。
- FAIL: 没有 `ws` 切换日志，后续 IRQ 路由一定不可靠。

Test 2: OP-TEE 中触发非安全 IRQ44, OP-TEE 直接丢弃

执行:

```
nanhu_irq_test trigger-ns
```

预期日志:

```
plic-sec: mpxy switch ws -> 1 ...  
[optee-test] trigger ns irq44 begin  
plic-tee claim: ctx=1 hart=0 mode=S irq=44 sec=0 ws=1  
[optee-plic] claim irq=44 sec=0  
[optee-plic] drop non-secure irq=44  
[optee-plic] complete dropped irq=44  
[optee-test] trigger ns irq44 end  
plic-sec: mpxy switch ws -> 0 ...
```

不允许出现:

```
riscv-plic: [ree-plic] claim cpu=0 hwirq=44  
[ree] irq... begin  
[ree] irq... end  
plic-sec: enter tee fiq irq=44
```

判定:

- PASS: IRQ44 由 OP-TEE S-context claim, 并在 OP-TEE PLIC 中 drop; 返回 REE 后 Linux 没有处理 IRQ44。
- FAIL: IRQ44 被 Linux claim, 说明没有在 OP-TEE 内 drop。
- FAIL: IRQ44 进入 OpenSBI M-context, 说明 PLIC 路由或 world_state 错。
- FAIL: OP-TEE 把 IRQ44 当 unhandled interrupt mask 掉, 说明 drop 逻辑放在 `interrupt_call_handlers()` 之后了, 位置错误。

Test 3: OP-TEE 中触发安全 IRQ45, OP-TEE 正常处理

执行:

```
nanhu_irq_test trigger-sec
```

预期日志:

```
plic-sec: mpxy switch ws -> 1 ...  
[optee-test] trigger secure irq45 begin
```



```
plic-tee claim: ctx=1 hart=0 mode=S irq=45 sec=1 ws=1
[optee-plic] claim irq=45 sec=1
[optee-test] secure irq45 handler begin
[optee-test] secure irq45 handler end
[optee-plic] complete secure irq=45
[optee-test] trigger secure irq45 end
plic-sec: mpxy switch ws -> 0 ...
```

不允许出现：

```
plic-sec: hart0 mctx=0 irq=45 ...
plic-sec: enter tee fiq irq=45 ...
riscv-plic: [ree-plic] claim cpu=0 hwirq=45
```

判定：

- PASS：IRQ45 由 OP-TEE S-context claim，OP-TEE IRQ45 handler 执行，OP-TEE complete。
- FAIL：IRQ45 进入 OpenSBI relay，说明 **ws=1** 或 PLIC 路由不对。
- FAIL：IRQ45 claim 后 unhandled/masked，说明 OP-TEE 没注册 IRQ45 handler。

对旧测试的影响

这个完整方案会改变系统的中断归类和 secure world 路由策略，因此需要重新确认旧测试：

42/43

预期仍应成立：

```
REE/Linux 下 IRQ42 正常给 Linux
IRQ42 handler 中触发 IRQ43
因为 ws=0 且 IRQ43 是 secure，IRQ43 仍走 OpenSBI relay
```

44/45 旧 hook 测试

旧 hook 仍可作为“OpenSBI relay 窗口内发生 NS IRQ”的对照测试，但它不再是新目标的主测试。

如果新路由设置为 **ws=1** 时 S-context 接收所有外部 IRQ，则 OpenSBI hook 触发 IRQ44 后，IRQ44 应该由 OP-TEE S-context claim 并 drop，而不是延后给 Linux。

因此旧 44/45 的预期需要更新为：

```
IRQ45 仍由 OpenSBI relay 进入 OP-TEE FIQ
OpenSBI hook 触发 IRQ44
IRQ44 不进入 Linux
OP-TEE claim/drop IRQ44
```

如果暂时不想让旧 44/45 受影响，建议给 OpenSBI hook 加编译宏，默认关闭：

- `opensbi/lib/utils/irqchip/fdt_irqchip_plic.c:72`

推荐提交拆分

建议按以下提交顺序做，方便回滚和定位：

1. OpenSBI：普通 MPXY enter/exit 同步 PLIC world_state。
2. QEMU：secure world 下外部 IRQ 统一递交 OP-TEE S-context。
3. OP-TEE platform：Nanhu 外部中断改 native，`interrupt_main_handler()` 接 PLIC dispatch，注册 IRQ45 handler。
4. OP-TEE PLIC：claim 后按 sec_src 分流，NS drop，secure call handler。
5. OP-TEE PTA：新增 `nanhu_irq_test` PTA。
6. REE CA/rootfs：新增 `nanhu_irq_test` 用户态工具。
7. 文档/脚本：新增 `trigger-ns`、`trigger-sec` 测试脚本和预期日志。

风险点和排查顺序

风险 1：PTA 期间 world_state 不是 1

现象：

```
trigger-sec 后仍出现 plic-sec: enter tee fiq irq=45
```

优先查：

- `opensbi/lib/utils/mpxy/fdt_mpxy_opteed.c`
- `fdt_plic_set_current_world_state(1/0)`
- QEMU `ws_csr`

风险 2：外部中断仍被 OP-TEE 当 foreign interrupt

现象：

```
PTA 触发 IRQ 后 OP-TEE 返回 REE/RPC foreign interrupt
```

优先查：

- `config/plat-nanhu/platform_config.h`
- `PLAT_THREAD_EXCP_FOREIGN_INTR`
- `PLAT_THREAD_EXCP_NATIVE_INTR`

风险 3：NS IRQ 没被 OP-TEE claim

现象：

```
trigger-ns 后没有 optee-pllc claim, 也没有 Linux handler
```

优先查:

- `qemu/hw/intc/sifive_pllc.c` 的 `pllc_irq_allowed()`
- OP-TEE 是否 enable S external interrupt
- OP-TEE PLIC S-context enable 是否打开 IRQ44/45 所在 word

注意: 如果 OP-TEE 要 claim/drop IRQ44, 也必须 enable IRQ44 在 OP-TEE S-context。否则 PLIC 不会递交给 OP-TEE。可以在 OP-TEE init 中 enable IRQ44 和 IRQ45, 其中 IRQ44 无 handler, 只在 PLIC drop 分支处理。

风险 4: secure IRQ45 被 claim 但无 handler

现象:

```
EMSG("Mask unhandled interrupt pllc:45")
```

优先查:

- IRQ45 handler 是否注册
- `interrupt_enable(chip, 45)` 是否执行
- `pllc_it_handle()` 是否在 `sec=1` 时调用 `interrupt_call_handlers()`

风险 5: NS IRQ44 被 mask

现象:

```
EMSG("Mask unhandled interrupt pllc:44")
```

说明 drop 逻辑位置错了。NS IRQ44 必须在 `interrupt_call_handlers()` 之前被识别并 complete, 不能让 OP-TEE generic interrupt layer 当成 unhandled IRQ。